Team Delivery Package: Project Management & System Integration

Enterprise Milestone-Based Technical Specifications

MODULE NAME:	Team 1: Project Management & System Integration
GIT BRANCH:	feature/project-management
LEAD MEMBERS:	Mohamed Gharieb, Mohamed Abd Elkhalek
WORKLOAD TIME:	4 weeks
KICKOFF DATE:	11 July 2026
CODE FREEZE:	23 July 2026
VERSION:	v1.2.0

1. Cover Page & Metadata Summary

This delivery package establishes the contract specifications for the **Team 1: Project Management & System Integration**.

Milestone	Target Date	Status	Description
Project Kickoff	11 July 2026	COMPLETED	Milestone scope locked & repository structured.
Code Freeze	23 July 2026	IN PROGRESS	All functional packages and tests locked.
Integration Phase	24 July 2026	PLANNED	Multi-team pipeline joins and bug resolution.
Final Presentation	25 July 2026	PLANNED	Final delivery and platform presentation.

2. Team Overview & Assignments

Resource Details

Lead Team Members: **Mohamed Gharieb, Mohamed Abd Elkhalek**

Branch Space: **`feature/project-management`**

Role Assignment: **Lead Systems Architects & Project Managers**

Difficulty Rating: **High** | Priority Index: **Critical** | Est. Workload: **4 weeks**

3. Business & Technical Goals

Business Objective

Align the 13-member developer team to deliver an integrated automated analytics product.

Technical Objective

Maintain system build compatibility, Docker environments, pipeline orchestrator configurations, and testing coverage checks.

Position inside AutoAnalyst AI

Root Orchestrator (pipeline.py) and CI/CD gatekeepers for all branches.

4. System Architecture & folder layout

Assigned folder Tree Structure

```
AutoAnalyst-AI/
■■■ src/autoanalyst/
■ ■■■ __init__.py
■ ■■■ pipeline.py
■ ■■■ utils/
■     ■■■ __init__.py
■     ■■■ helpers.py
■■■ .github/workflows/
    ■■■ ci_cd.yml
```

Target Code Modules

Your codebase modifications belong inside: `src/autoanalyst/pipeline.py, src/autoanalyst/utils/helpers.py`.

Functional Code Interface Specifications

```
# src/autoanalyst/pipeline.py
from dataclasses import dataclass, field
import pandas as pd
from typing import Any, Literal

@dataclass
class PipelineConfig:
    target_column: str | None = None
    model_task: Literal["classification", "regression", "auto"] = "auto"
    missing_strategy: str = "median"
    encode_categoricals: bool = True
    model_test_size: float = 0.2
    random_state: int = 42

@dataclass
class PipelineResult:
    raw_df: pd.DataFrame
    cleaned_df: pd.DataFrame
    model_ready_df: pd.DataFrame
    profile: dict[str, Any]
    missing_values_report: pd.DataFrame
    insights: list[str] = field(default_factory=list)
    model_results: dict[str, Any] | None = None
    evaluation_results: dict[str, Any] | None = None

def run_analysis_pipeline(
    dataset: str | pd.DataFrame,
    config: PipelineConfig | None = None
) -> PipelineResult:
    """Orchestrates the execution of all team submodules."""
    pass
```

API Specification

```
Input Parameters:
- dataset: Union[str, pd.DataFrame] - Path to a CSV/Excel or loaded dataframe.
- config: PipelineConfig - Setup configuration values for preprocessing/modeling.

Output Returns:
- PipelineResult: Data structure holding dataframes, dictionary metrics, and insight strings.
```

5. Git Workflow & Code Integration Policy

Branching Workflow

All developers pull and write inside `feature/project-management` created from the latest `develop` branch.

Pull Request Requirements

1. Merge latest develop branch changes into features before raising PRs.
2. Assign team lead reviews and resolve conflicts.
3. A minimum of one approved review check is required.

6. Quality Standards & Guidelines

Format & Lint Rules

All variables, methods, and file layouts must use lower_case snake_case parameters. Code comments must be in English.

Error Checking & Imputers

All functional blocks should be guarded by custom exception checks (raise ValueError/KeyError).

Logging Rules

Incorporate logging warnings rather than std print statements.

7. Expected Deliverables & Verification

Required Files list

`src/autoanalyst/pipeline.py, src/autoanalyst/utils/helpers.py`

Verification Code Assertions

Unit test validations are situated in: `tests/test_pipeline.py`.
Pipeline integration assertions belong in: `tests/test_end_to_end_pipeline.py`.

IMPORTANT GUIDELINE
All functional checks must compile successfully with zero syntax warnings before code freeze.

8. Definition of Done Checklist

- [ ] All functions compile cleanly with PEP 8 checks.
- [ ] Unit tests cover all key methods and functions.
- [ ] Try-except checks isolate external dataframe issues safely.
- [ ] The PR has been reviewed and verified by Team 1.

9. Common Mistakes & Troubleshooting

Pitfalls to Avoid

- Omitting import sanity checks, causing compile crashes.
- Adding custom pandas operations that bypass sub-team helper functions.
- Allowing direct commits to develop or main without code review approvals.